

Lab 3 - EECS 211

Winter 2026

1 Overview

In this lab, we are going to implement a simple OS scheduler. This OS scheduler will cycle through available processes, executing or resuming them from their ready or yield states, respectively, until all have finished.

Lab environment: All tasks in this lab are performed on EECS-provided Linux servers.

Readings and references: Students are encouraged to consult the following resources for additional background:

- How to Use Inline Assembly Language in C Code - Official GNU Compiler Collection (GCC) online documentation about how to embed assembly language instructions inside C code.

Rubrics (100 points): The code template is given on Canvas. Implement the scheduler described above to switch between these three processes. You will upload one zip file that contains:

- **PDF report:** This includes a screenshot of the messages displayed in the terminal and a brief explanation of your implementation choices.
- The code template folder (**lab3** folder) after filling out your implementation.

The execution order will be the following:

1. **Process1** starts running, prints a message (**process1 is running**), and then yields.
2. **Process2** starts running, prints a message (**process2 is running**), and then yields.
3. **Process3** starts running, prints a message (**process3 is running**), and then yields.
4. **Process1** resumes from the point that was yielded, prints a message (**process1 is back from yield**), and then exits.
5. **Process2** resumes from the point that was yielded, prints a message (**process2 is back from yield**), and then exits.
6. **Process3** resumes from the point that was yielded, prints a message (**process3 is back from yield**), and then exits.
7. Print a message (**All done**) and halt the program.

Note: TODOs in `scheduler.c` are weighted at 20 points each and all other TODOs are worth 15 points each. Points will be deducted if no screenshots or sufficient explanations are given. The students should also **fill the honesty statement and upload it to canvas**, otherwise 30 points will be deducted.

Due date: 02/18/2026, 23:59:59 PST

2 Implement a Simple Scheduler

To simplify the implementation, we will make the following assumptions:

- The maximum number of processes that can be loaded in the memory is equal to 3.
- The OS will keep track of the state of each process using three status variables: `state1`, `state2`, and `state3`.

These variables can have three different values depending on their status. A value of 0 means that the corresponding process is ready to run, a value of 1 means that the process has terminated, and a value of 2 means that this process is in the yield state.

Variable	Value	Meaning
	0	The process is ready to run
	1	The process has terminated
	2	The process is in the yield state

Table 1: Status variables and their meanings

The scheduler begins by invoking the trap handler, then sequentially checks the state variables mentioned above. Once it finds a process that is ready to execute or in the yield state, it will jump to the corresponding line of code in the process.

- When the process yields (by using the `yield_()` function), the OS will change the status of this process to 2 (yield) and make another `ecall` to go back to the entry of the trap handling procedure.
- Similarly, when the process exits (by using the `exit()` function, see the template `stdlib.h`), the OS will change the status of this process to 1 (terminated).
- If no process needs to run, then the scheduler will enter an infinite loop.

The main body of this simple OS (where the CPU starts executing) needs to initialize the values in the state variables, determine which process should be executed next, and set the starting address of the process to be executed next. If there are both ready state and yield state processes, the ready state processes are prioritized for execution first.

3 Template Code

The goal of this lab is to implement the yield functionality of the processes. The template code only supports exit functionality, and the process cannot stop its execution in the middle of the task with the current implementation.

In order to support it, you need to add the `yield_()` function and some logic to check the process states in the scheduler. Note that when resuming a process from the yield state, you need to start execution at the next line of code that was executed before it yielded.

The template code consists of source codes in a mix of C and assembly. The overall workflow of this template is as follows:

1. `main.c` initializes some functions and makes an `ecall` to enter the scheduling.
2. `trap_entry.s` stores current register values into the stack and jumps to the trap handler.
3. `trap.c` calls the scheduler (In general, many different procedures are defined here and called depending on the reason for the interrupt).
4. `scheduler.c` determines which process to execute next and sets the starting address of the process in `mepc`.

5. Return to `trap_entry.s` to load back the register values and jump to the address by `mret` that is set in the previous step.
6. Execute the process and then yield/exit it.
7. Go back to step 2 and repeat the steps until all the processes are terminated.

You have TODOs in the following files, so you can implement the yield functionality by simply filling them out:

- `scheduler.c`
- `processes.c`
- `stdlib.h`